Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:		Semester:	1st sem
Name of the Student:	A. Nikhil Koushik	Roll Number:	A24126552128
Name of the Lab:	Full Stack	Timings:	9:40:1210

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	2	4	5	2	13	10	
8/7	2	4	4	5	1	14	10	
15/7	3	4	4	5	1	14	10	
22/7	4	4	4	4	1	13	10	
29/7	5	4	4	4	1	13	10	
5/8	6	4	4	5	2	14	10	
9/8	7	4	4	4	1	13	10	
16/9	8	4	3	4	2	13	10	
23/9	9	4	4	4	1	13	10	
30/9	10	4	4	4	1	13	10	

Name of the Faculty: Ch. Swathi

Faculty Signature:

INDEX

S.No.	ANITS Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1	8/7/26	Introduction to HTML and CSS	CO1		
2	15/7/26	Static Webpage using HTML and CSS	CO1		
3	22/7/26	HTML codes using audio & Vedio Tags.	CO1		
4	29/7/26	Program using javascript	CO1		
5		Dynamic Webpage using javascript	CO1		
6	5/8/26	Implement Modules in Nodejs	CO1		
7	12/8/26	Develop mini application using Express & Node js.	CO1		
8	19/08	Code Tantra	CO1		
9	2/09	Code Tantra.	CO1		
10	9/9	Code Tantra	CO1		
11.	16/9	Functional & Class Component	CO2		
12	23/09	HTTP's using json server	CO2 / CO3		
13	30/9	mongo DB			

Introduction to HTML and CSS.

⇒ HTML (HyperText Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, photographs, links, images, list, tables forms and other multimedia content. HTML documents are interupted by web browsers to display the content in a standard / structured and meaning-ful manner. HTML provides the structure and semantic organisation of webpage.

⇒ CSS (Cascading style sheets) is a style sheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colours, fonts backgrounds, spacing, borders, alignment, dimensions and layouts CSS also supports responsive design, allowing web pages to adopt to different screen sizes and devices CSS can be applied using inline, internal or External style sheet.

→ HTML and CSS work together to develop Webpages, where HTML provides the structure and content while CSS provides the styling, layout Visual Representation. The combination of both forms the foundation of modern web. development and Enables the creation of organized, attractive, and user-friendly web intifaces.

Differences between MERN Stack and Full Stack

→ MERN Stack is a javascript based technology stack used for developing mordern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js MongoDB is used for data Base Management, Express.js handles the Backent Frame work and Api's, React.js is used to develop Faront End user interface and Node.js provides the Server side runtime Environment.

→ Full Stack Developement refer to the developement of Both the Fontend and Backend of a web application along with data Base integration. A Full Stack developer works with technologies used for designing the user interface, implementing Sever-side logic Managing data Base

HTML components:

1. HTML Tags - Define the structure and elements of a web page
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> Tag.
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag.
6. Lists - used to create ordered and unordered list using <ol>, <ul>,
7. Tables - used to organise data in rows and columns.
8. Forms - used to collect user input using Element such as text boxes, buttons and Check Boxes.
9. Div and span - used to group and organise web - page content
10. Sematic Elements: Elements such as <header> <nav>, <section>, <article>, <footer>

Provide meaningful web page structure.

CSS components:

1. Selectors - selects HTML elements to apply styles
2. properties - Define the visual characteristic of elements
3. Values - specify the setting of a CSS property
4. Colors - Define text, background, border colo
5. Font - Control Font family, size, style and weight
6. Backgrounds - set Background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - Control's the space outside an Element
9. Padding - controls the space inside Element
10. layout - controls the arrangement of elements using Flex box, grid and positing.

HTML VS HTML5

HTML	HTML5
older version of HTML	latest major version of HTML
limited multimedia suppst	suppsts audio and video.
used External plugins for multimedia	No External plugins needed
Has fewer Sematic tags.	provides sematic Tags.
No <Canvas> Element	suppst <Canvas>
Limited Web storage	suppsts local storage
Complex DOCTYPE declaration	simple <!DOCtype HTML>

CSS VS CSS3.

CSS	CSS3.
Earlier version of CSS	Advanced version of CSS.
Basic styling features	provides advanced styling features
limited animations	suppsts animations and Transitions
No Flexiable layout systems.	suppsts Flexbox and Grid
limited visual Effect	suppsts shadows, gradient Effects
No multimedia queies	suppsts media queries
uses a single specification	Divided into different modules

1) Design an HTML code using audio Tags

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favourite song </h2>
    <audio controls>
        <source src = "song.mp3" type = "audio/mp
        your Browser does not not support the audio tag
    </audio>
</body>
</html>
```

o/p: Display an audio player with play pause and volume controls.

Step s:

Create a new HTML file
Add the Basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with control attribute
Specify the audio file using <source> tag save and open the HTML file in a web browser.
play the audio using displayed controls.

ii) Design a HTML code using video Tags.

Code:

```
<!DOCTYPE html>
<html>
 <head>
      <title> Video Example </title>
 </head>
 <body>
      <h2> My video </h2>
      <video width = "400" controls>
          <source src = " video.mp4" type = "video.
                                              mp4">
      </video>
 </body>
 </html>
```

O/p: Display a video player with play, pause, volume and Full screen controls.

Steps:

Create a new HTML file.
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with control attribute
specify the video file using <source> tag
Save the file and open it in web browser
play the video using displayed controls

Design a static webpag using HTML and CSS.

pomodoro Timer, using HTML, CSS

```
<!DOCtype html>
<html>
<head>
    <title> pomodoro Timer </title>
    <style>
        body {
            font-family : Arial ;
            back-ground : #f5f5f5 ;
            text-align : center
            padding : 50px
        }
        .timer {
            width : 300px ;
            margin : auto ;
            padding : 20 px
            background : White ;
            border-radius : 15px ;
            box-shadow : 00 10px gray ;
        }
        h1 {
            color : #e74c3c ;
        }
        .time {
            font-size : 50px ;
            margin : 20px ;
        }
```

```
button {
    padding : 10px 20px ;
    margin : 5px ;
    border : none ;
    border-radius : 5px ;
    background : #e74c3c;
    color : white ;
    }
</style>
</head>
<body>
    <div class = "Timer" >
    <h1> pomodoro Timer </h1>
    <div class = "time" > 25:00 </div>
    <button > start </button >
    <button> Reset </button >
    </div >
</body >
</html >
```

o/p:

Write a program using java script Carrays, functions

Code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11]

function displayArray(arr) {
    console.log("Array Element:");
    for (let i=0; i<arr.length; i++) {
        console.log(arr[i])
    }
}

function findSum(arr) {
    let sum = 0;
    for (let i=0; i<arr.length; i++) {
        sum += arr[i]
    }
    return sum;
}.
```

Experiment - 04

Develop Dynamic Web page using javascript

program:

```
<!Doctype html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body {
            text-align : center;
            font-family : Arial;
            margin-top : 100px;
        }
        button {
            padding : 10px, 20px;
            font-size : 16px;
            cursor : pointer;
        }
    </style>
</head>
<body>
    <h1 id = "heading"> Welcome to my webpage </h1>
    <p id = "message"> Click the button to change
        this page </p>
    <button onclick = "changepage()"> click me
        </button>
```

```
<script>
    function changepage() {
    document.getElementById("heading").
        innerHTML = "Welcome to javascript!";
    document.getElementById("message").
        innerHTML = "The Webpage has been changed
        dynamically";
    document.body.style.backgroundcolor = "lightblue"
    }
</script>
</body>
</html>
```

output:

Welcome to my webpage

click the button to change this web page

click me

- When user clicks "click me", the javascript function changepage() is called, then the heading dynamically changes and Background also changes

After clicking the button;

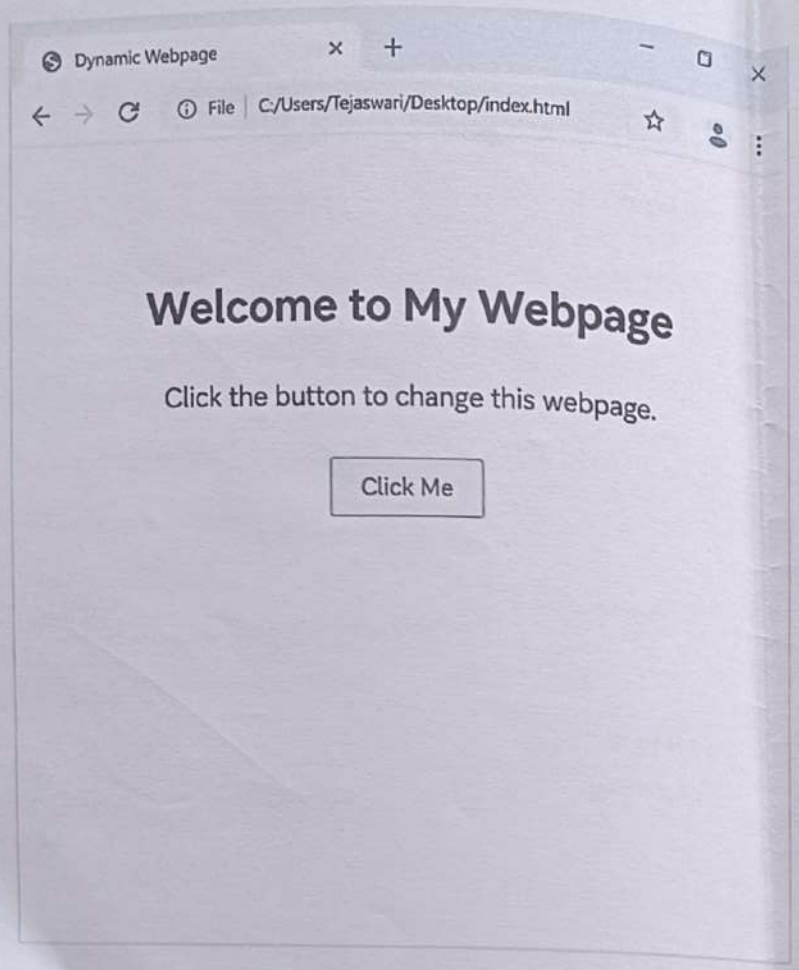
Welcome to javascript!

This Webpage has been dynamically changed

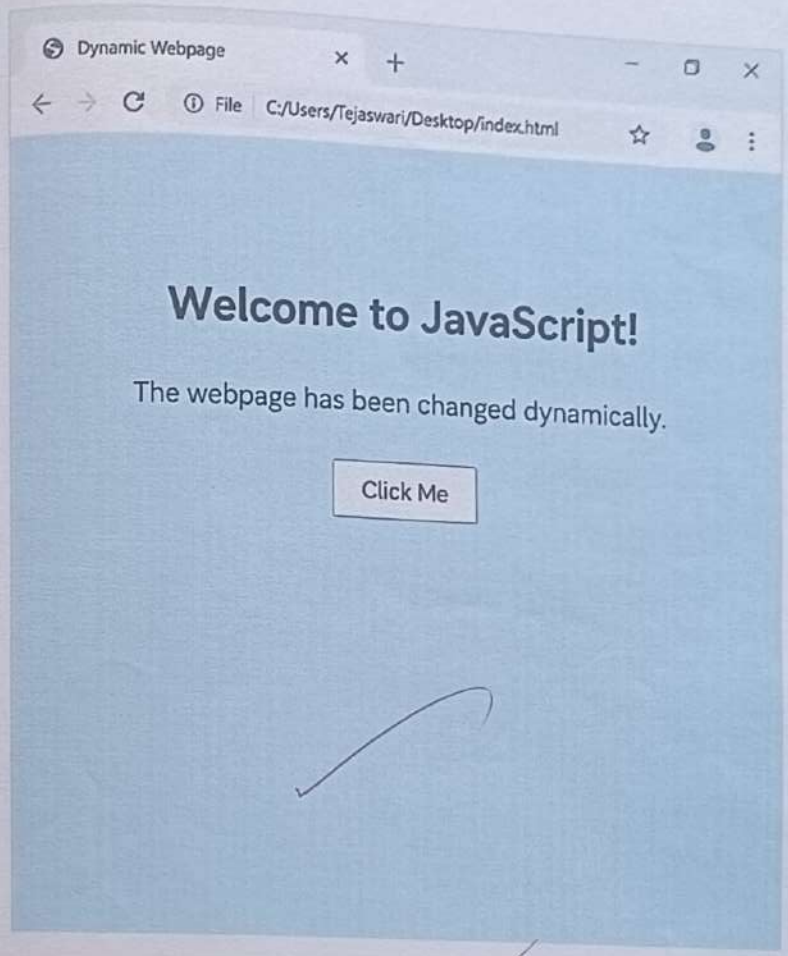
click me

<script>

function changepage() {



Before clicking the button



After clicking the button

Experiment-05

Implement Modules in Node.js

In Node.js, you can implement modules using two distinct systems: common JS (CJS), which is traditional default and Es module (ESM), the modern JavaScript standard.

1. The common JS Method (default):

common JS uses modules exports to share code and require() to import it this system load modules synchronously.

a) Create the module (math.js)

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
        add,
        subtract
};
```

b) use the module (app.js)

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

output : 15
3.

2. The Es modules method: (modren standard)

Es modules use Export to share code and important to use it. To make Node.js recoginize Es modules. you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Creati the module (calculator.js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b

const calculatorName = "supercali";
export default calculatorName;
```

b. we the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';

console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

Output:

```
supercali
8
2
```

Using Built in core modules:

Node js also comes equipped with dozens of built in core modules (like fs for file system or path for file routing) that don't need installation we will load them by module Names:

* // CommonJS style:

  Const fs = require ("fs")

  // Es modules style.

  'import fs From 'fs';

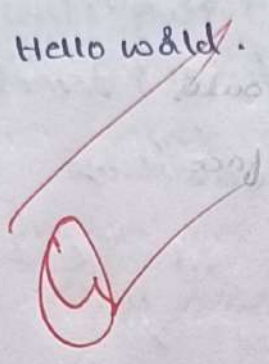
Here are some of the most Commonly ones:

1. fs = file system operations.
2. http = http server and client
3. path - file path utilities
4. os = operating system utilities
5. events = Event handling
6. util - utility functions
7. Stream - Stream handling
8. Crypto - Cryptographic functions
9. Url - URI pasing
10. querystring - URI query string handling

Example : Simple HTTP server.

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

Output : Hello world.



Experiment - 06

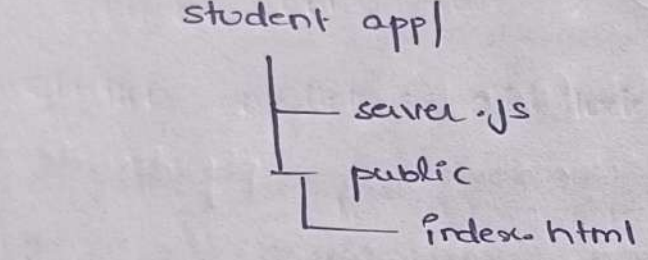
Develope mini application using express and Node js

Simple mini application using Nodejs and Express js

Student Management System

1. Add a student
2. View all students
3. delete a student
4. Uses Express js routes
5. Simple HTML interface

- Project structure:



Server.js

```
Const express = require ("express");
Const app = express ();
Const PORT = 3000;
let students = [];
app.use (express.json());
app.use (express.static ("public"));
app.get ("/students", (req, res) => {
    res.json (students);
});
```

```
// Add
app.post("/students", req, res) => {
    const student = {
        id = students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    student.push(student);
res.json({ message: "student added succesfully",
        student: student });
});
// Delete
app.delete(`/students/:id`, (req, res) => {
const id = pause int(req.params.id);
Student s = students.filter(student => student.id!=
=id);
res.json({
    message: "student deleted succesfully"});
});
// start server.
app.listen(PORT, () => {
    Console.log('server running at http://localhost:
    ${PORT}`);
});
```

public / index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> student Management </title>
</head>
<body>
    <h1> student Management </h1>
    <input type = "text" id = "name" placeholder =
        " Enter student name" >
    <input type = "text" id = "course" placeholder =
        " Enter course" >
    <button onclick = "addStudent()" > Add Student
                                    </button>
    <h2> Student List </h2>
    <ul id = "studentList" > </ul>
    <script>
        function addStudent() {
            let name = document.getElementById("name
                                    ).value;
            let course = document.getElementById("course").
                                    value;
            fetch("/students", {
                method: "POST",
                headers = {
                    "Content-Type": "application/json"
```

```
body: JSON.stringify({
    name: name,
    course: course,
  })
})
.then(response => response.json())
.then(data => {
    alert(data.message);

document.getElementById("name").value = "";
document.getElementById("course").value = "";

displayStudents();
});
}

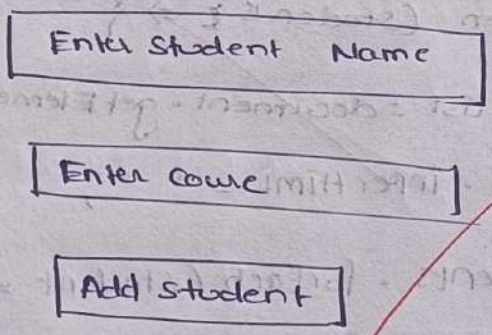
function displayStudents() {
  fetch("/students").then(response => response.json())
  .then(students => {
    let list = document.getElementById("studentList");
    list.innerHTML = "";

    students.forEach(student => {
      let li = document.createElement("li");
      li.innerHTML = student.id + " - " +
                     student.name + " - " +
                     student.course + "
      <button onclick="deleteStudent(" + student.id + ")">Delete</button>";
      list.appendChild(li);
    });
  });
}
```

```
function deleteStudent(id) {
    fetch("/student/" + id, { method: "DELETE" })
    .then(response => response.json())
    .then(data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

Output:

Student Management System



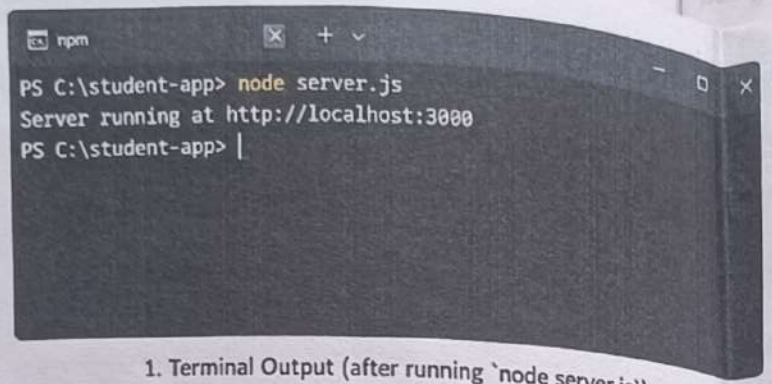
Student list

1. Nikhil - CSM [delete]
2. Ram - CSM [delete]

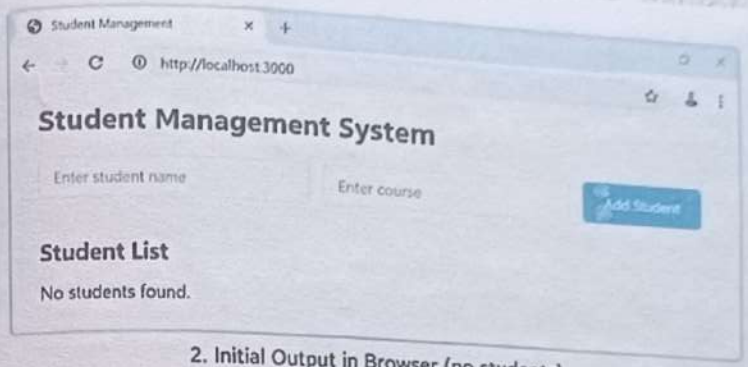
- The Express.js server is successfully started
  http://localhost:5000

The Browser displayes the student management system with fields

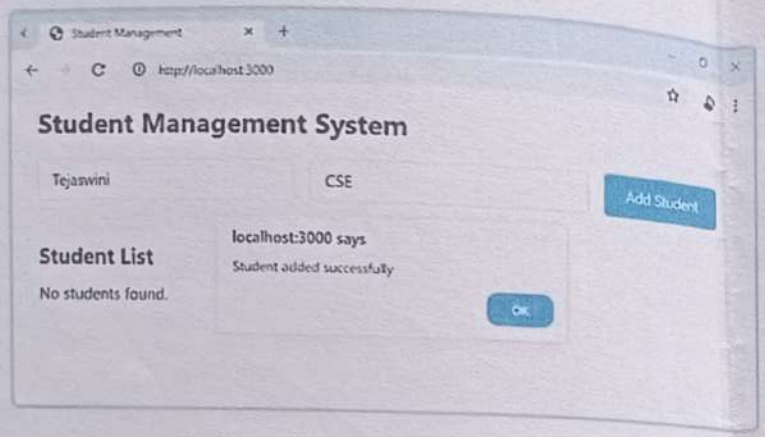
.then (response => response.json())



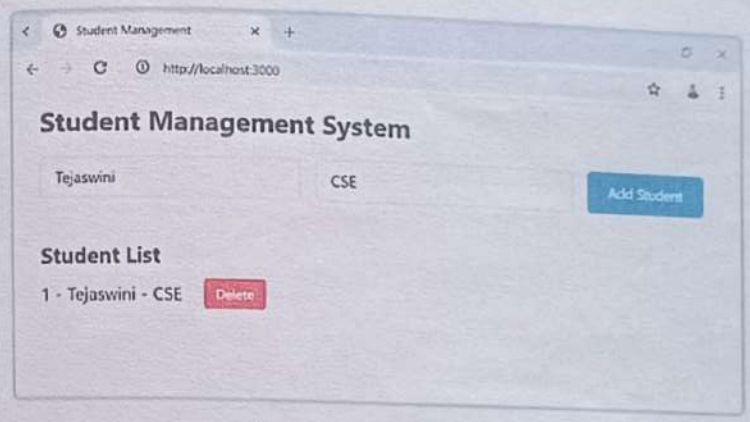
1. Terminal Output (after running `node server.js`)



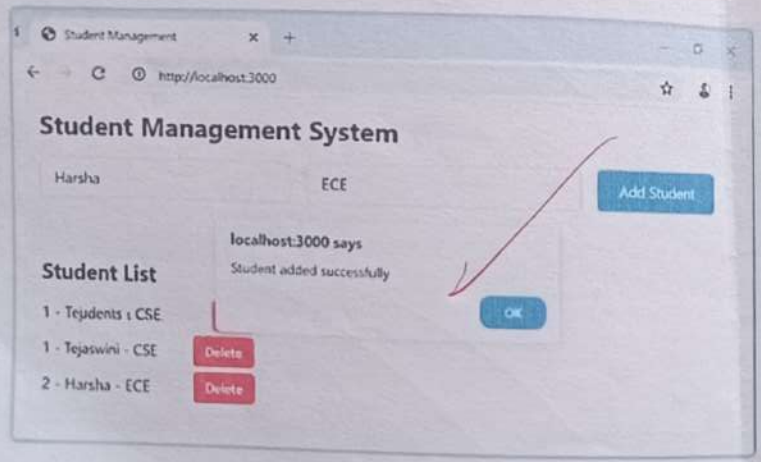
2. Initial Output in Browser (no students)



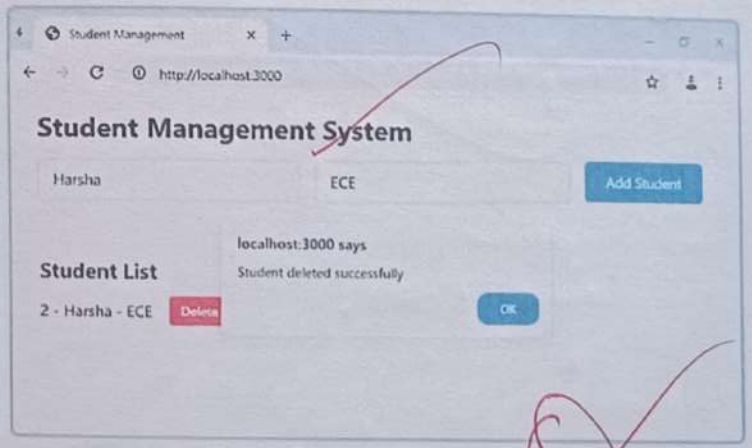
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

Create a functional component and class component using react

Aim: To develop a react application using functional Component and class Component to display student details, Manage state, handle button clicks and dynamically update data on a webpage.

functional Component:

A functional component is a java script function that returns to display the user interface at uses react hooks such as usestate to manage state and update the webpage dynamically.

program:

```
import React, {useState} from
'react';
function Student-function() {
    Const [name, setName] = Use state ("Nikhil");
    Const [course, setCourse] = use state ("CSM");
    Const [Count, Set Count] = Use state (0)
    Const change Name = () => {
        Set Name ("Nikhil");
    };
    Const Change Course = () => {
        Set course ("CSD");
    };
    Const incrementCount = () => {
        Set-Count (Count + 1);
    };
    return (
```

```
<div style = {{
    border : "5px solid blue",
    padding : "20px",
    margin : "20px",
    borderRadius : "10px",
    textAlign : "center",
}}>
<h1> StudentDetails </h1>
<h2> Functional Component </h2>
<p> <b> Name : </b> {name} </p>
<p> <b> Course: </b> {course} </p>
<p> <b> Button Clicks </b> {count} </p>
<button onClick = {changeName} >
        Change Name </button>
<button onClick = {changCourse} >
        Change Course </button>
<button onClick = {incrementCount} > increment </button>
</div>
};
```

Output: When the user clicks the Buttons, the Name, course and count change dynamically without reloading the webpage.

Class Component:

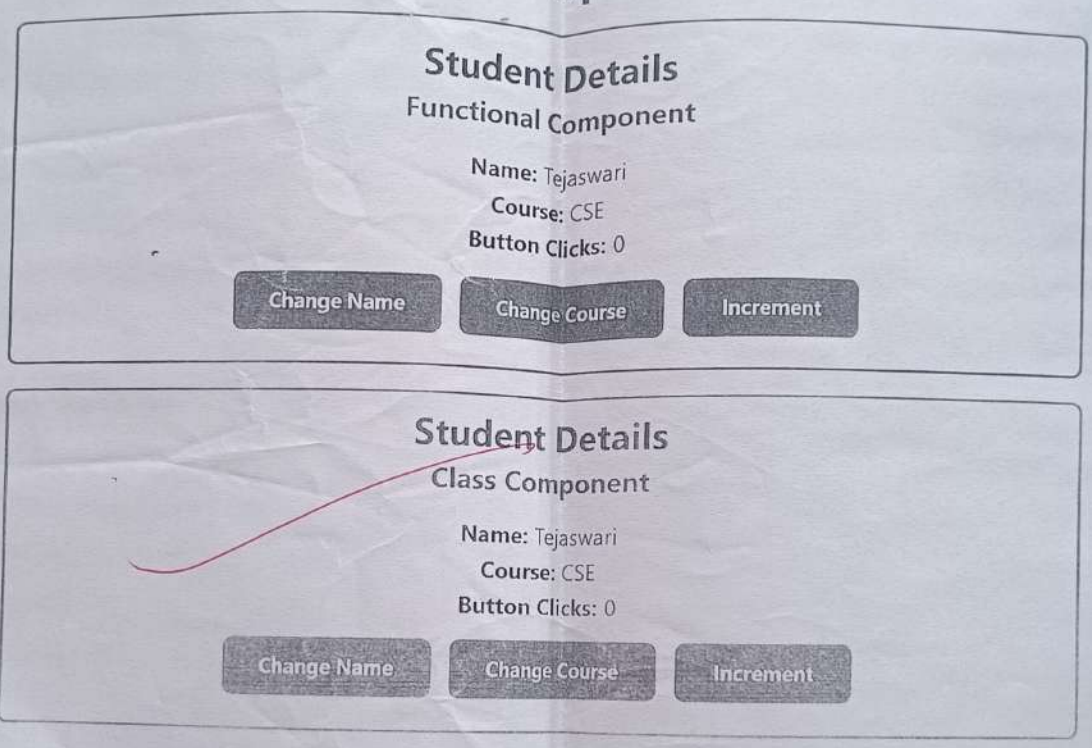
A class component is an ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

program:

```
import React, { Component } from
'react';
class Student Class extends Component {
    constructor (props) {
        super (props);
        this.state = {
            name = "Nikhil"
            course = "CSM"
            count = 0
        };
    }
Change Name = () => {
    this.state ({
        name = "Nikhil"
    });
};
Change Course = () => {
    this.setState ({
        course = "AI&ML"
    });
};
increment count = () => {
    this.setState ((prevState) => {
        count: prevState.count + 1
```

```
    }));
  };
render() {
  return (
    <div style={{
        border: "2px solid green";
        padding: "20px"
        Margin: "2px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1> Student Details </h1>
    <h2> Class Component </h2>
    <p> <b> name: </b> {this.state.name} </p>
    <p> <b> Course: </b> {this.state.course} </p>
    <p> <b> Button clicks </b> {this.state.count} </p>
    <button onClick={this.changeName}>
                Change name </button>
    <button onClick={this.changeCourse}>
                Change Course </button>
    <button onClick={this.incrementCount}>
                increment </button>
    </div>
```

React Components



Experiment-08

implement http methods using json server

Aim: To implement HTTP method such as GET, POST, PUT, patch and Delete using json server to perform CRUD (Create, Read, update, Delete) operations on student records

Theory: JSON server is a tool used to create a fake Rest API using a JSON file. it allows us to perform HTTP operations without developing a backend server From Scratch

HTTP method without developing a backend server From Scrach.

HTTP method, are used to communicate b/w a client and a server

- GET - Retrives data From the server
- POST - add's new data to the server
- PUT - Replaces an existing record
- patch - Update selected field of a record
- Delete - Removes a record From the screen.

a. Create a json server.

Step2: initilize the project

open the vs code terminal and Execute the following Commands.

```
mkdir JSON server
cd JSON server
npm init -y
npm install JSON server
```

Step2 : Create db.json

Create a file named db.json and add the following student data.

```
{ "student" :[
    {
      "id" :"1",
      "name" : "Nikhil",
      "course" : "CSM",
      "marks" : "90",
    },
    {
      "id" : 2 ,
      "name" : "Ravi",
      "course" : "CSE",
      "marks" : "100" ;
    }.
```

Step3 : Start the JSON server

For the current JSON server version, execute

npx json-server db.json

The server start at :

http://localhost:3000

The student records can be access using

http://localhost:3000/student

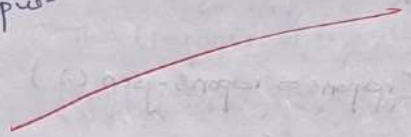
2. Implementation of HTTP method

a. GET method: The GET method is used to retrive student records from json server.

Code:

```
Fetch ('http://localhost:3000/students')
.then (response => response.json())
.Then (data => {
        console.log(data);
    })
.catch (catch => {
        console.log(err);
    });
```

* output-



5. post method: The post method is used to insert a new student record into the server.

```
code: Fetch ("http://localhost:3000/students", {
        methods: "post",
        header: {
          'Content-Type': 'application/json'
        },
      body JSON.stringtype ( {
          name: "vishno";
          course: "csm",
          marks: "95",
      })
    })
  Then (respons => respons.json())
  Then (data => {
        Console.log (data);
    })
  .catch (err => {
        Console.log (err);
    });
```

Output:

C. put method: The put method is used to replace an Exisisting student record with updated data

```
Code: Fetch("http://localhost:3000/students/1", {

    method: "put",
    header: {
        'Content-Type': "application/json",
    body: json-stringify({
        name: "Nikhil",
        Course: "CSM",
        Mark: "95",
    })
})
.then(response => response.json())
.then(data => {console.log(data);})
.catch(err => {console.log(err);});
```

output:

5. PATCH method: The PATCH method is used to update only selected field as an existing student record.

Code: Fetch ("http://localhost:3000/student/9", {

method: "PATCH",

header {

'content-Type': "application/json" }

]

body: json.stringfy ({

marks: 99

})

})

then (respons => response.json())

Then (data => {

console.log (data); });

Then (err => {

console.log (err); });

});

output;

E. Delete Method: The delete method is used to remove an existing student record from the server side.

```
Fetch("https://localhost:3000/", {
    method: "Delete"
})

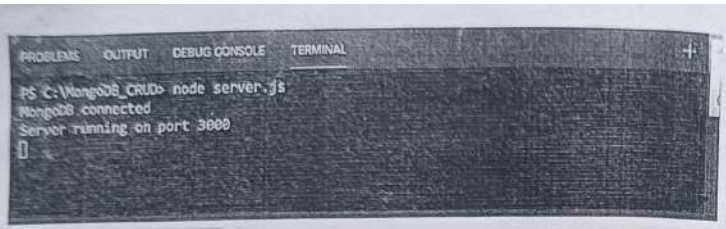
Then (response => {
    if (respons.ok) {
        console.log ("student deleted
                    succesfully");
    }
})
catch (err => {
    console.log (err);
});
```

output:

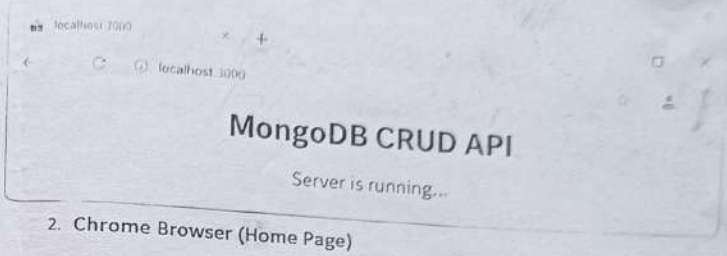
· After executing the post, put, patch and delete operation in sequence the remaining student will be

output:

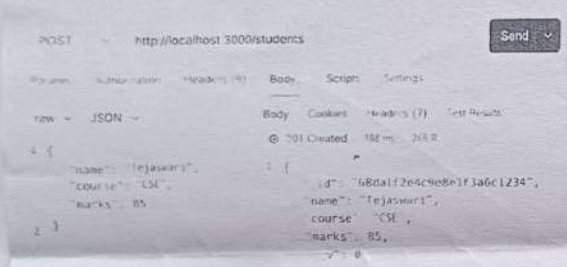
Thus, HTPS method GET, POST, PUT, PATCH, DELETE were sucessful implemented using json server The CRUD server operation are performed on student records The data was retrived, inserted, updated and deleted using HTTP requested.



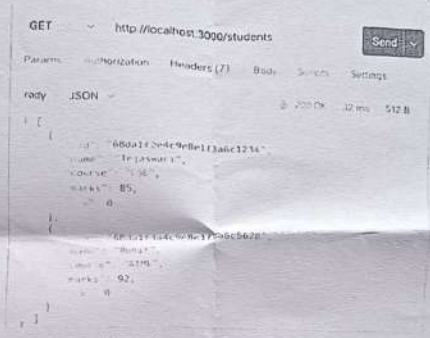
1. Terminal Output



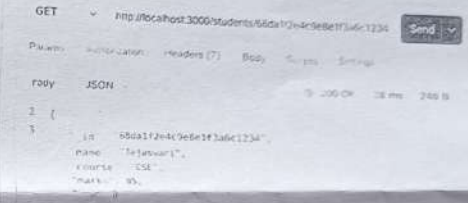
2. Chrome Browser (Home Page)



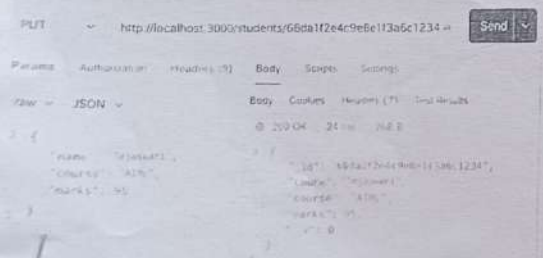
3. Create (POST) – Add Student



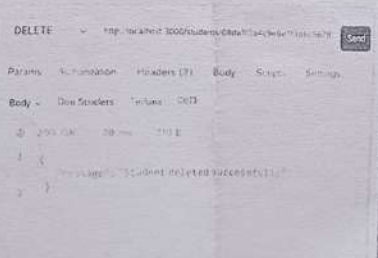
4. Read (GET) – Get All Students



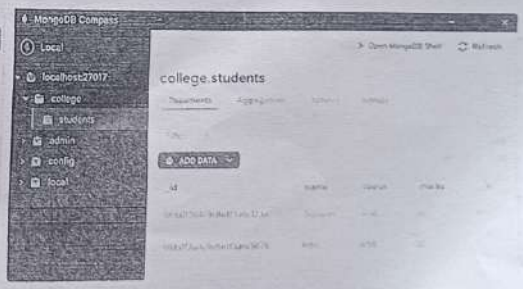
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View

Experiment - 9

Develop CRUD operations using Mongodb.

Aim: To develop a web application using node.js, Express.js and Mongo.DB to perform CRUD operation on student Records.

1. installation and project setup:

```
mkdir mongoDB - CRUD
cd. mongoDB - CRUD
npm init - y
npm install Express mongoose.
```

2. program server.js

```
Const Express = require ("Express")
Const mangoose = require ('mangoose');
Const app = express();
Const PORT = 3000;

mongoose.Connect ("mongooredb://127.0.0.1:2701/collage")
then (() => console.log ("mangoDB Connected "))
Catch (err => Console.log (err));

Const student schema = new. mangoose.
schema {
    name : {
        type : string
        require : true
    }.
```

```
Course : {
    type : string
    required : true
    }
Marks : {
    type : number
    required : true
  }
});

Const student :
        mongoose.model ("student", student, schema)

app.post ("/student", async (req, rest. => {
Const student = new. student (req. stud|);
Const save student = await student sowo ();
res. stalu |201 ). json (sowe . students ;
} catch (err8 ) {
    res. stalu (400 ) .json ({ merage: err
                                merage} ) ;
  }
});

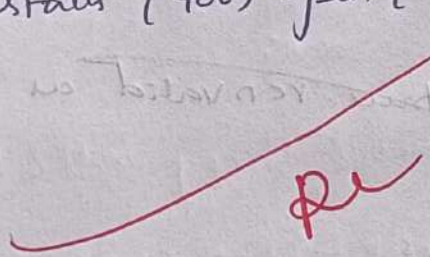
app . get ("/student , async (req, rest ⇒ {
  try {
  cust student = await student . find ()
  res . stalu (sow . json ({ merage : err
                          merage() ) ;
  }
});
```

```
app.get("/student/:id", async (req,res) =>
  {
  try{
  const student = await student.findBYID(req.
                  param.id);

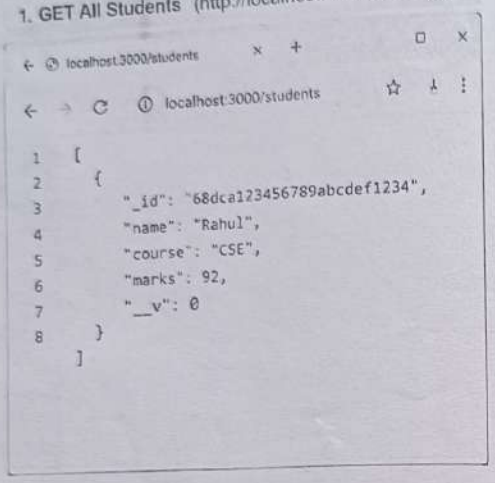
if (!student){
  return res.student(400).json{
      message "student not found"
  });

app.put("student/id.async(req,res) => {
    try {
    const student = await student.findBYID and
    update(
      req.paramsid
      req.body
    {
    new.true.runValidators: true
  });
if(!student){
    return res.status(400).json({
        message:'student not found"
    });
  }.
res.status(200).json(student);
  } catch(err){
res.status(400).json({message: err.message});
}
```

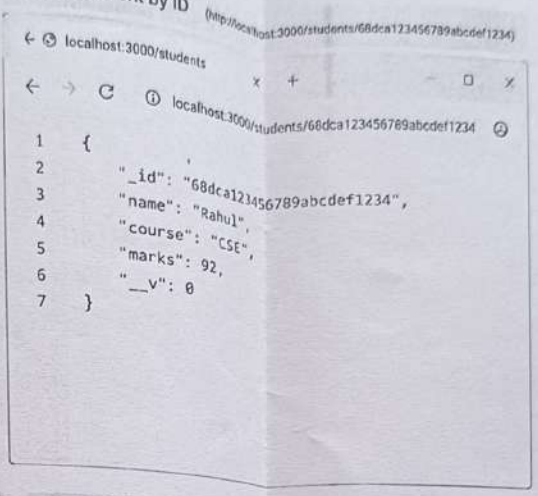
```
app.delete("/student/id : async (req,res) => {
    try{
    const student = await student.findById
    and delete(req.param.id);
    if(!student){
        return res.status(404).json({
            message:"student not found"
        });
    }
    res.status(200).json({
        message:"student deleted sucessfully
    });
    }
    catch(err){
        res.status(400).json({message.err.mes
    }
});
```



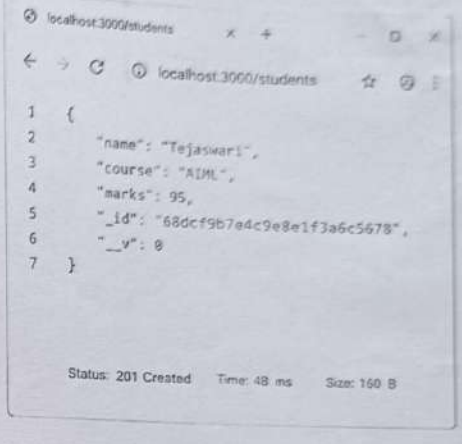
1. GET All Students (http://localhost:3000/students)



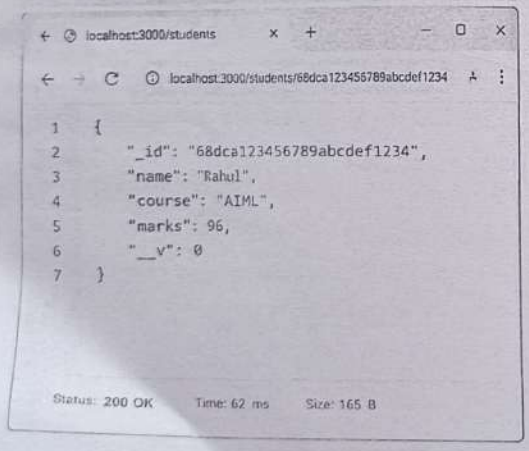
2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



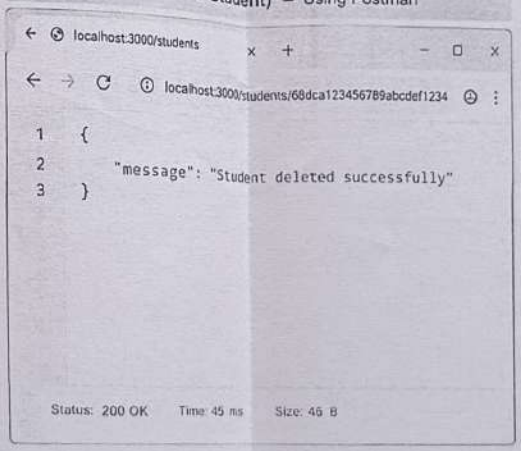
3. POST (Create Student) – Using Postman



4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)

